데이터 분석을 위한

Numpy

류영표
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

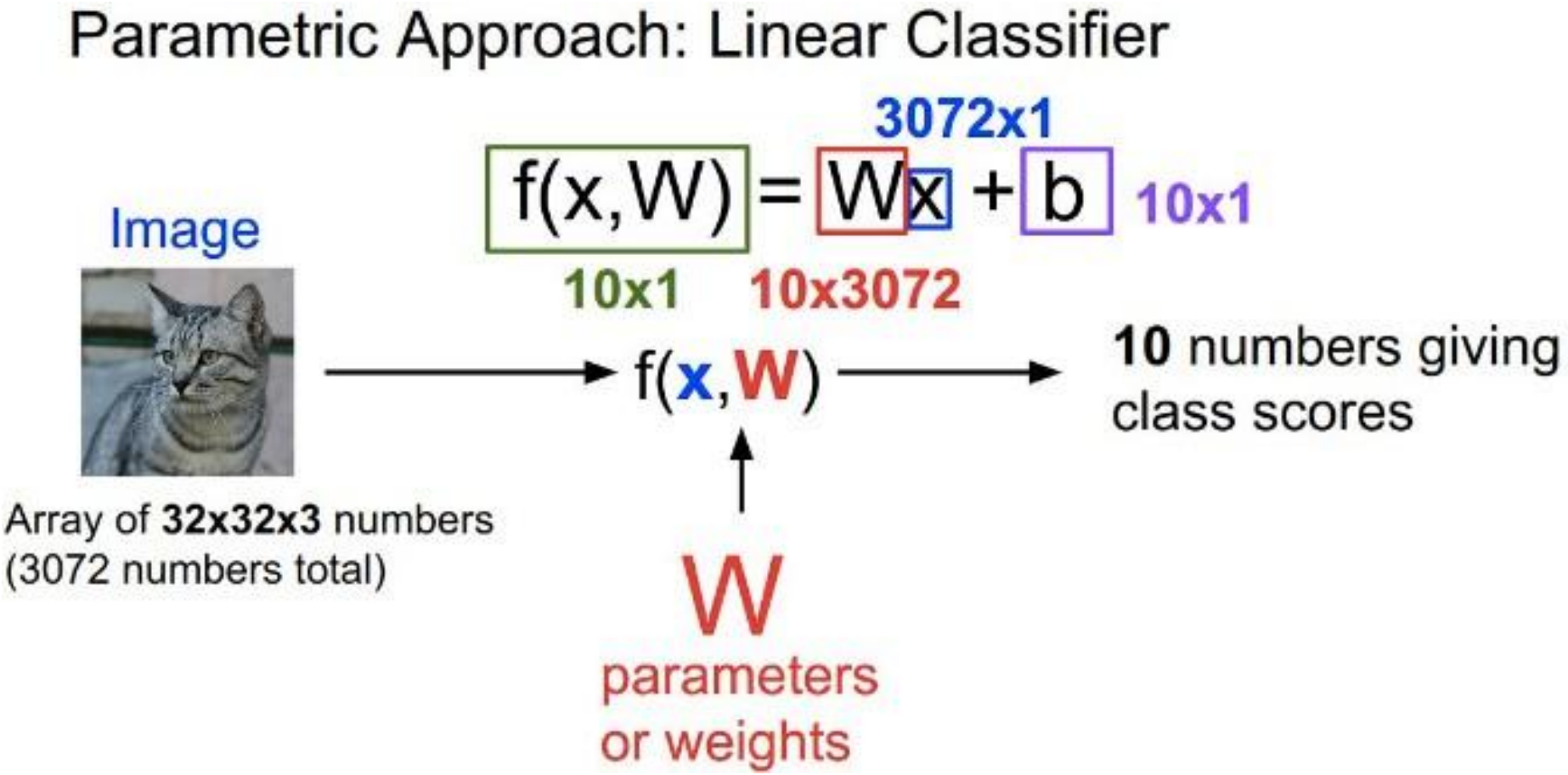
- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

Numpy

Numerical Python의 약자로 산술계산용 라이브러리



http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture02.pdf

Numpy가 왜 필요한가?

문제	Numpy의 역할
Python list 반복 연산은 느리다	벡터화 연산으로 수십배 빠름
2D 이상 배열 표현이 어렵다	다차원 배열 ndarray를 제공
데이터 과학, ML 연산이 복잡	고성능 수치 계산 라이브러리
Pandas, Tensorflow와 연결 필요	모든 AI, 데이터 라이브러리의 기반

- Numpy의 핵심 기능
  - 내부적으로 C기반으로 구현되어 매우 빠름
  - 다차원 배열 객체 ndarray
  - 효율적인 메모리 구조와 브로드캐스팅
  - 선형대수, 통계, 난수, 변환 등 수치 계산

Ndarray VS List

```
import numpy as np, time
lst = list(range(1_000_000))
arr = np.array(lst)

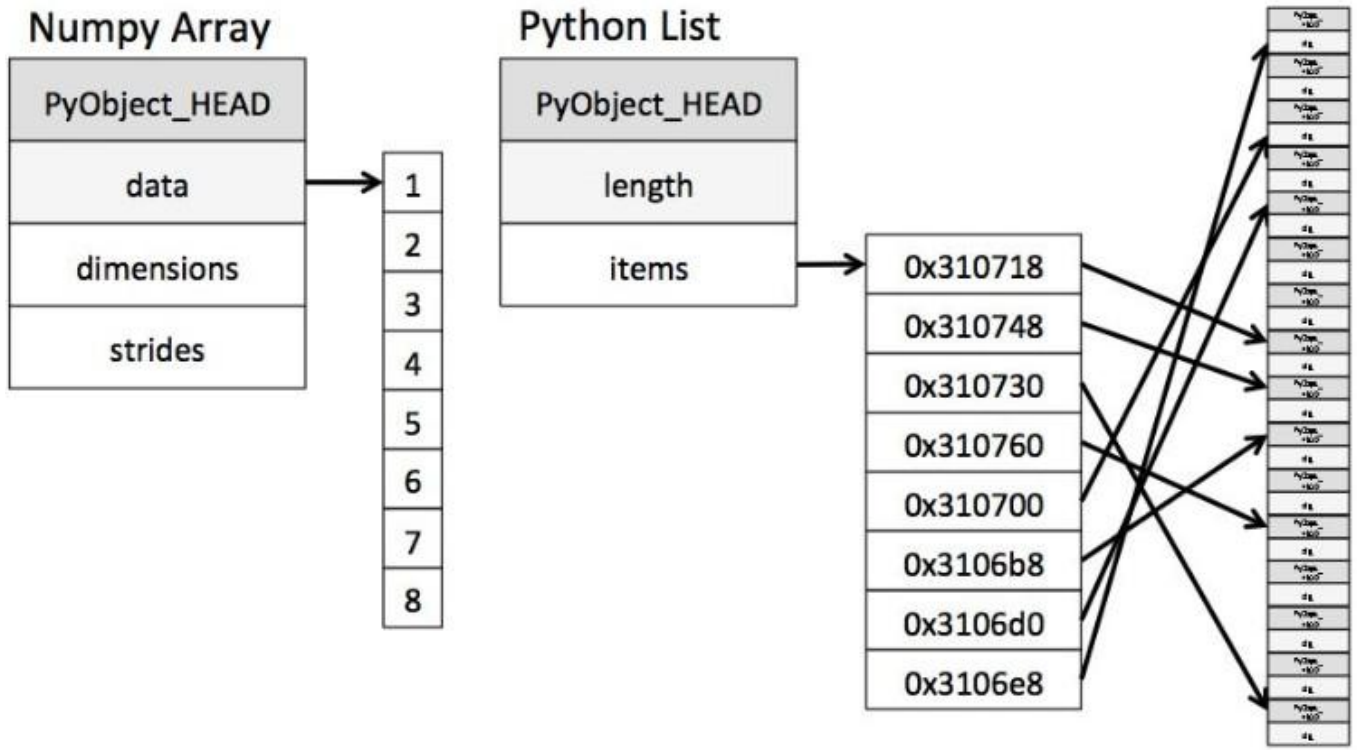
start = time.time()
sum([x**2 for x in lst])
print("List:", round(time.time()-start,3),"초")

start = time.time()
np.sum(arr**2)
print("Numpy:", round(time.time()-start,3),"초")
```

```
List: 0.116 초
Numpy: 0.003 초
```

Ndarray

- ndarray vs. list
  - 연속된 메모리 블록에 데이터를 저장
  - 같은 종류의 데이터를 담음



출 처 : https://jakevdp.github.io/blog/2014/05/09/why-python-is-slow/

ndarray

```python
import numpy as np

# List 예시
print("List 예시")
L = [1, 2, 3]
print("초기 L:", L)

# += 대신 append 사용
L.append(4)
print("append(4) 실행 결과:", L)

# 리스트 확장은 +=로 가능
L += [10, 20, 30]
print("L += [10, 20, 30] 실행 결과:", L)
print()

# Numpy ndarray 예시
print("Numpy ndarray 예시")
A = np.array([1, 2, 3])
print("초기 A:", A)

# ndarray는 원소 단위로 += 가능 (브로드캐스팅)
A += 1
print("A += 1 실행 결과:", A)
print()

# 내부 구조 비교
print("List는 객체의 참조를 저장")
print("각 원소의 id:", [id(x) for x in L])

print()
print("Numpy ndarray는 연속된 메모리 블록에 값 저장")
print("데이터 버퍼 주소:", A.__array_interface__['data'])
```

```
List 예시
초기 L: [1, 2, 3]
append(4) 실행 결과: [1, 2, 3, 4]
L += [10, 20, 30] 실행 결과: [1, 2, 3, 4, 10, 20, 30]

Numpy ndarray 예시
초기 A: [1 2 3]
A += 1 실행 결과: [2 3 4]

List는 객체의 참조를 저장
각 원소의 id: [11654376, 11654408, 11654440, 11654472, 11654664, 11654984, 11655304]

Numpy ndarray는 연속된 메모리 블록에 값 저장
데이터 버퍼 주소: (197673328, False)
```

ndarray

- ndarray는 각 원소별로 동일한 데이터 타입으로 처리.
  : 값들이 동일한 데이터 타입을 갖는 그리 형태의 데이터 구조

array([원소 , 원소 , 원소 , dtype=])

```
import numpy as np

A = [1,2,3,4]

a = np.array(A, np.int)
print(type(a))
print(a)
```

```
<class 'numpy.ndarray'>
[1 2 3 4]
```

```
b = np.array(A,np.str)
print(type(b))
print(b)
```

```
<class 'numpy.ndarray'>
['1' '2' '3' '4']
```

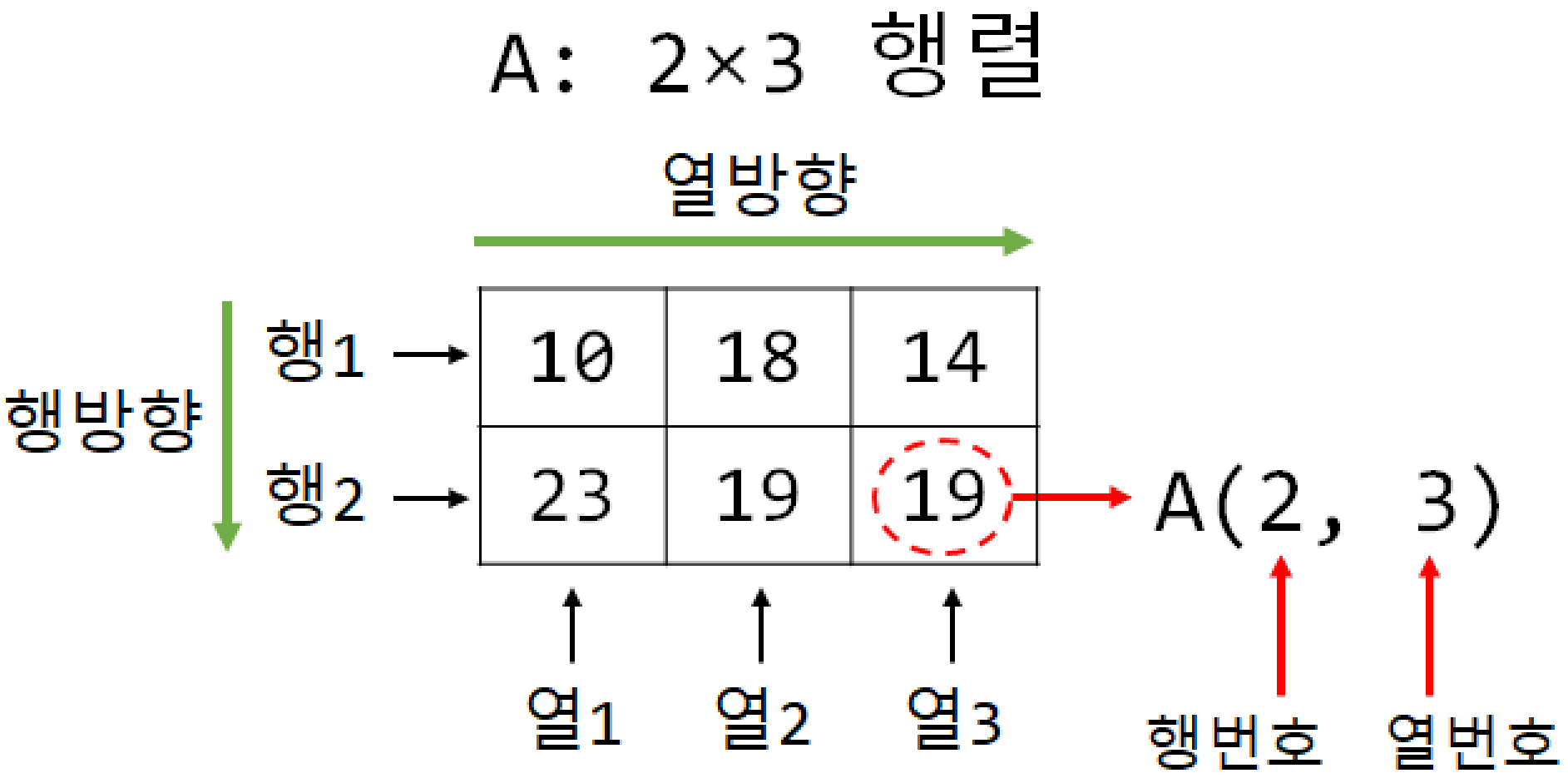
```
c = np.array(A,np.float)
print(type(c))
print(c)
```

```
<class 'numpy.ndarray'>
[1. 2. 3. 4.]
```

```
c = np.array(A,np.complex)
print(type(c))
print(c)
```

```
<class 'numpy.ndarray'>
[1.+0.j 2.+0.j 3.+0.j 4.+0.j]
```

행렬(Matrix)



ndarray

- 검색이나 슬라이싱(slicing)은 새로운 배열을 생성하지 않고 원본을 참조

  -> 원본 데이터가 함께 변경될 수 있음.

  -> 변경 방지를 위해 array.copy() 메서드로 새로운 배열을 생성해야 함.

```
import numpy as np
A = [1,2,3,4]

a = np.array(A)
s = a[:2]
print('A의 출력입니다 : %a ' %a)
print('s의 출력입니다 : %s' %s)
```

```
A의 출력입니다 : array([1, 2, 3, 4])
s의 출력입니다 : [1 2]
```

```
ss = a[:2].copy()
print(ss.size)
ss[0] =99
print(a)
print(s)
print(ss)
```

```
2
[1 2 3 4]
[1 2]
[99  2]
```

ndarray

- **벡터화** : 배열은 for문을 작성하지 않고 데이터를 일괄 처리 하는 것
  - 같은 크기의 배열 간의 산술 연산은 배열의 각 원소 단위로 적용됨

```
[2]  arr = np.array([[1, 2, 3], [4, 5, 6]])
     arr
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
[3]  arr + arr
```

```
array([[ 2,  4,  6],
       [ 8, 10, 12]])
```

```
[6]  arr / arr
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
lit = [[1,2,3],[4,5,6]]
lit2 = lit

result = []

for i in range(len(lit)):
    temp = []
    for j in range(len(lit[i])):
        temp.append(lit[i][j]+lit2[i][j])
    result.append(temp)
print(result)
```

- 브로드캐스팅(**broadcasting**)
  - 두 배열의 **shape**을 오른쪽, 즉 마지막 축부터 비교합니다.
    - 각 축에서 다음 중 하나를 만족하면 브로드캐스팅이 가능

    1. 두 축의 크기가 같다
    2. 둘 중 하나의 축 크기가 1이다.
    3. 한쪽 배열에 해당 축이 없다.

```
(2, 3)
(   3)
```

```
(2, 3)
(2, 1)
```

```
(2, 3)
(2, 2)
```

산술 연산

- 브로드캐스팅

1) 스칼라 인자 : 모든 원소에 각각 적용

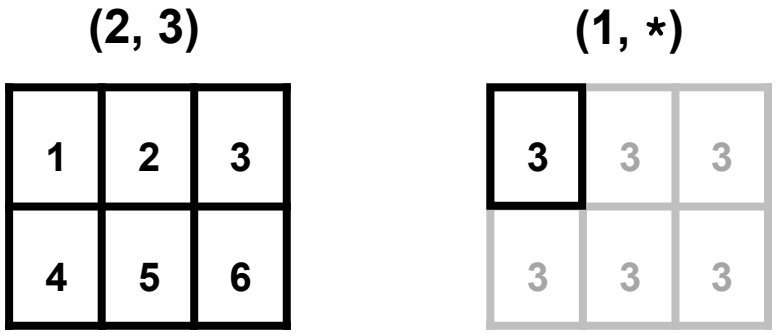
```
[9] 10 - arr
```

```
array([[9, 8, 7],
       [6, 5, 4]])
```

```
[10] arr*3
```

```
[11] arr/3
```

```
array([[0.33333333, 0.66666667, 1.
       [1.33333333, 1.66666667, 2.
```



산술 연산

- 브로드캐스팅

2) 배열

```
[15] arr

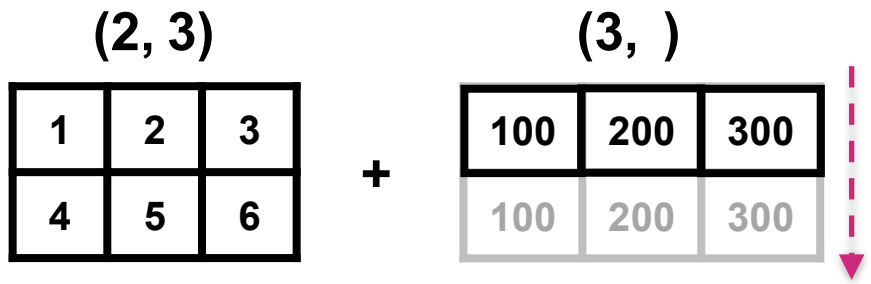
     array([[1, 2, 3],
            [4, 5, 6]])

[36] arr2

     array([100, 200, 300])

[37] arr + arr2

     array([[101, 202, 303],
            [104, 205, 306]])
```



산술 연산

- 브로드캐스팅

2) 배열

```
[15] arr

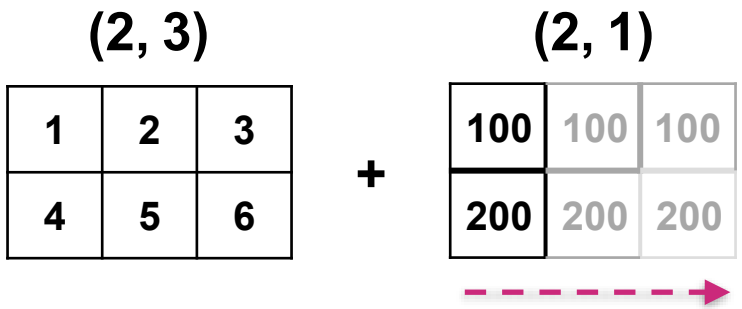
     array([[1, 2, 3],
            [4, 5, 6]])

[21] arr3

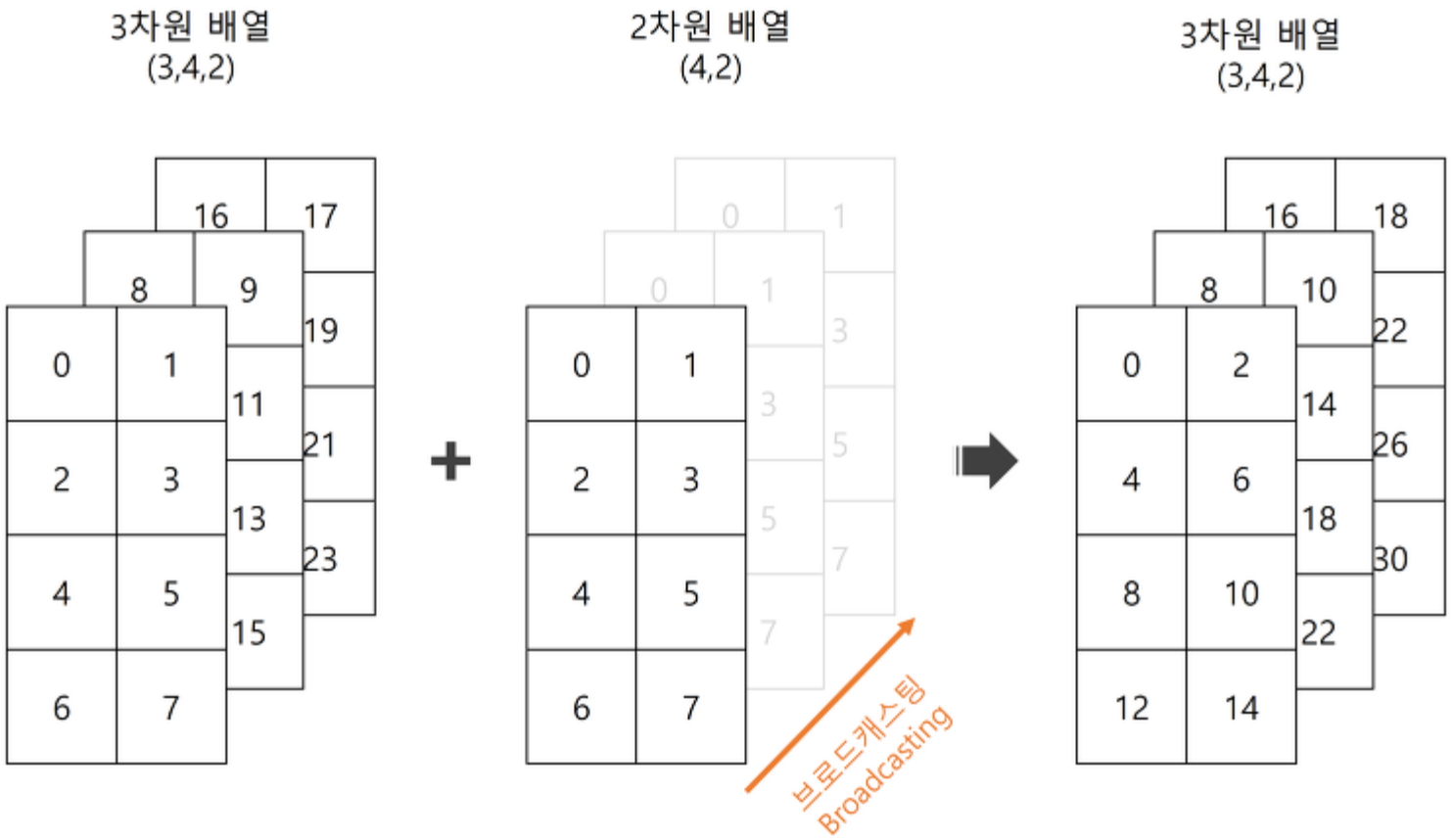
     array([[100],
            [200]])

     arr + arr3

     array([[101, 102, 103],
            [204, 205, 206]])
```



broadcasting



출처 : https://sacko.tistory.com/16

ndarray attributes

1. dtype

- 배열에 담긴 원소의 자료형(ndarray는 같은 자료형을 담음)

```
[7]  arr = np.array([10, 20, 30, 40])
     arr
```

```
array([10, 20, 30, 40])
```

```
[8]  arr.dtype
```

```
dtype('int64')
```

```
[10] arr2 = np.array([[0.1, 0.6],[-2, 6]])
     arr2
```

```
array([[ 0.1,  0.6],
       [-2. ,  6. ]])
```

```
arr2.dtype
```

```
dtype('float64')
```

ndarray attributes

1. dtype

- dtype으로 데이터 타입을 명시하지 않은 경우 자료형을 추론하여 저장
- dtype이 있어 C나 포트란 같은 저수준 언어로 작성된 코드와 쉽게 연동이 가능

```
[2]  arr = np.array([10, 20, 30, 40])
     arr
```

```
array([10, 20, 30, 40])
```

```
[3]  arr.dtype
```

```
dtype('int64')
```

```
[8]  float_arr = arr.astype(np.float64)
     float_arr
```

```
array([10., 20., 30., 40.])
```

```
[9]  float_arr.dtype
```

```
dtype('float64')
```

→ astype이라는 메소드를 이용하여
dtype을 명시적으로 변환할 수 있음

ndarray attributes

2. size

- 배열에 있는 원소의 전체 갯수

```
arr
```

```
array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
[12] arr.size
```

```
8
```

```
arr2
```

```
array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]])
```

```
[16] arr2.size
```

```
12
```

ndarray attributes

3. ndim

- 배열의 차원의 갯수

```
arr
```

```
array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
arr2
```

```
array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]])
```

```
[14] arr.ndim
```

```
1
```

```
arr2.ndim
```

```
2
```

Byte와 bit

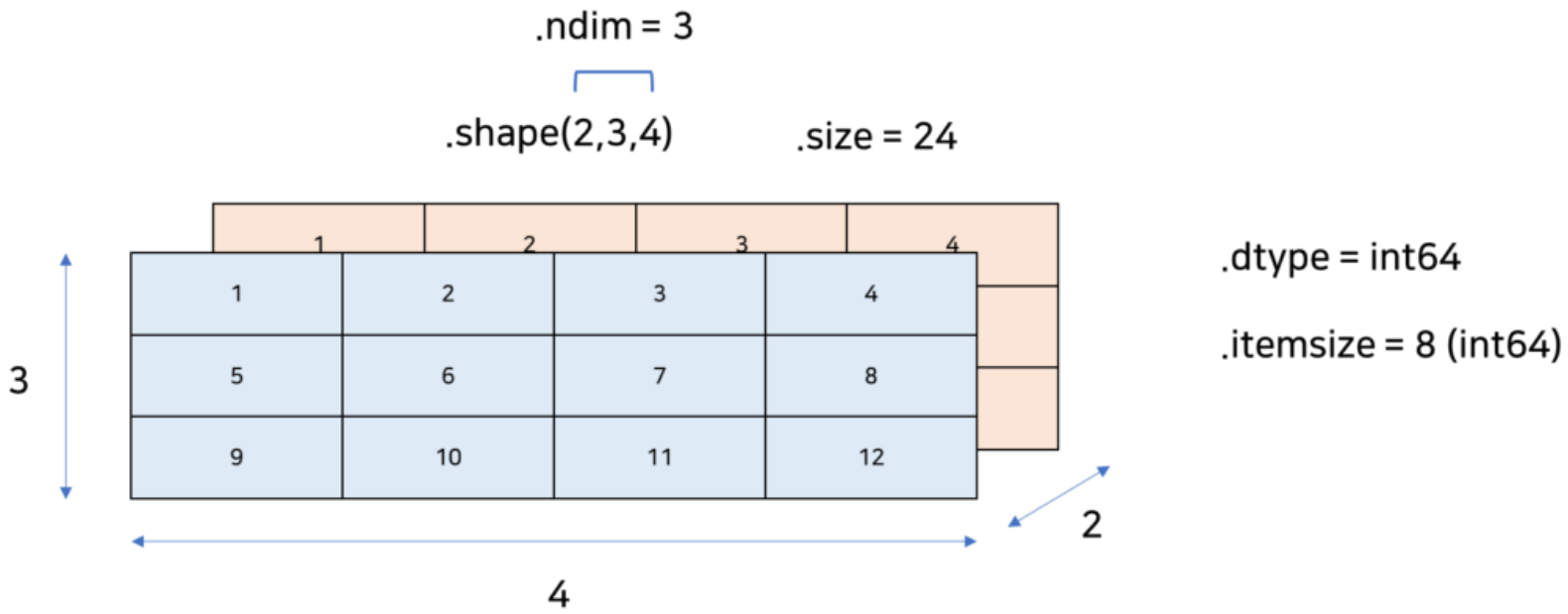
= 비트와 바이트 =

비트 (binary digit, bit)	바이트 (Byte)
0과 1, 두 가지 값만 가질 수 있는 측정 단위	여덟 개의 비트로 구성된 데이터의 양을 나타내는 단위
 0 OFF FALSE  1 ON TRUE	

삼성반도체이야기
samsungsemiconstory.com

ndarray attributes

- ndarray.ndim : 배열의 차원(축)의 개수를 나타냄.
- ndarray.shape : 배열의 모양을 나타내는 정수의 튜플. 각 차원에서 배열의 크기를 나타냄.
- ndarray.size : 배열의 모든 원소의 개수를 나타냄.
- ndarray.dtype : 배열에 저장된 원소의 타입을 나타내는 객체.
- ndarray.itemsize : 배열에 저장된 각 원소의 크기를 바이트 단위로 나타냄.



ndarray attributes

- ndarray.ndim : 배열의 차원(축)의 개수를 나타냄.
- ndarray.shape : 배열의 모양을 나타내는 정수의 튜플. 각 차원에서 배열의 크기를 나타냄.
- ndarray.size : 배열의 모든 원소의 개수를 나타냄.
- ndarray.dtype : 배열에 저장된 원소의 타입을 나타내는 객체.
- ndarray.itemsize : 배열에 저장된 각 원소의 크기를 바이트 단위로 나타냄.

```
import numpy as np

my_list = [[[1,2,3,4],[5,6,7,8,],[9,10,11,12]],
           [[1,2,3,4],[5,6,7,8],[9,10,11,12]]]

my_array = np.array(my_list)
print('차원 : ', my_array.ndim)
print('모양 : ', my_array.shape)
print('크기 : ', my_array.size)
print('데이터 타입 : ', my_array.dtype)
print('각 원소의 크기 : ', my_array.itemsize) #배열의 각 요소의 크기를 바이트 단위로 반환 (int64는 8바이트 정수)
```

```
차원 :  3
모양 :  (2, 3, 4)
크기 :  24
데이터 타입 :  int64
각 원소의 크기 :  8
```

ndarray attributes

3. 0차원

스칼라 값을 np.array()에 전달하면 shape이 ()인 0차원 ndarry가 생성됨.

Column : 열

Row : 행

[0]

```
import numpy as np
a = np.array(10)
print(a)
print(a.ndim)
```

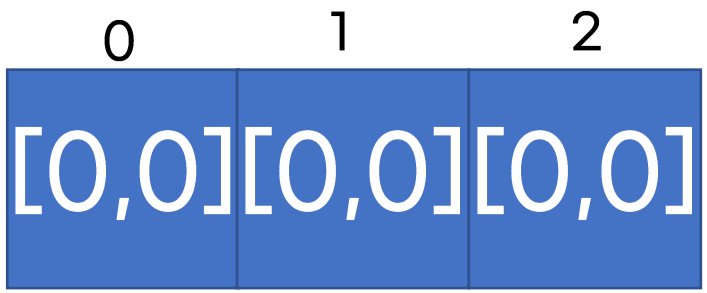
```
10
0
```

ndarray attributes

3. 1차원

배열의 특징. 차원,형태, 요소를 가지고 있음.

생성시 데이터와 타입을 넣으면 ndim(차원)으로 확인.




```python
import numpy as np
a = np.array([1,2,3])
print(a)
print(a.ndim)
```

```
[1 2 3]
1
```

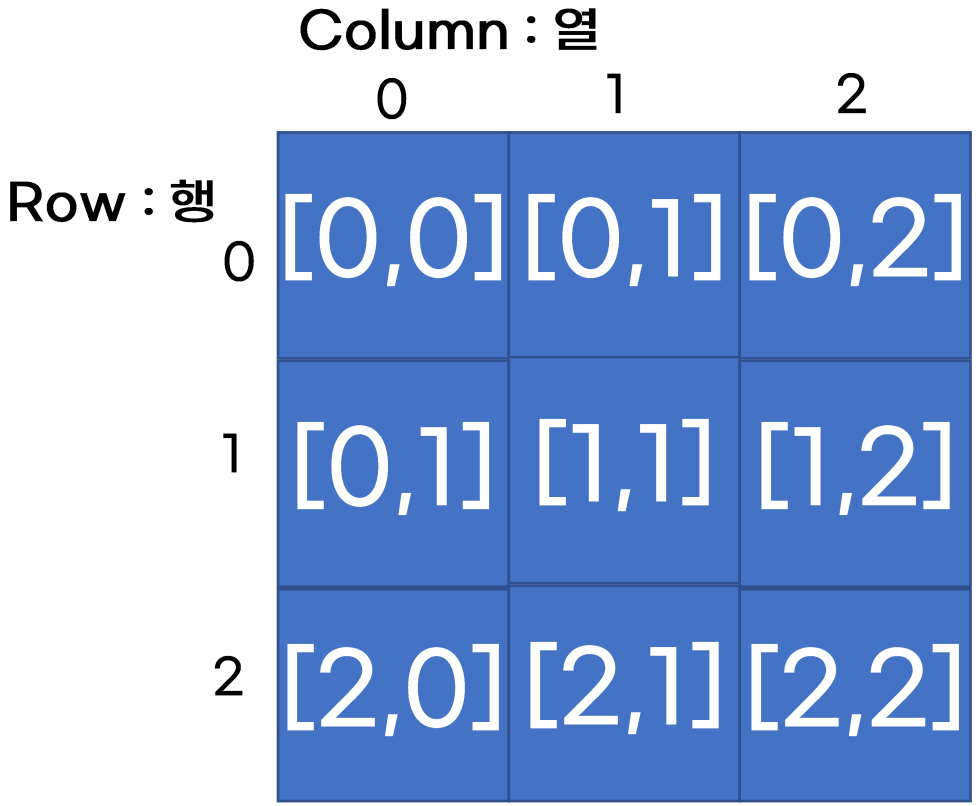
1차원 배열

ndarray attributes

3. 2차원

배열의 특징. 차원,형태, 요소를 가지고 있음.

생성시 데이터와 타입을 넣으면 ndim(차원)으로 확인.




```
import numpy as np
a = np.array([[1,2],[3,4]])
print(a.ndim)
```

```
2
```

```
import numpy as np
a = np.array([ [1,2],[3,4],[4,5],[6,7],[8,9] ])
print(a.ndim)
```

```
2
```

ndarray attributes

3. 3차원

numpy.array 생성 시 sequence 각 요소에 대해 접근변수와 타입을 정할 수 있음.

1-D array

7	2	9	10

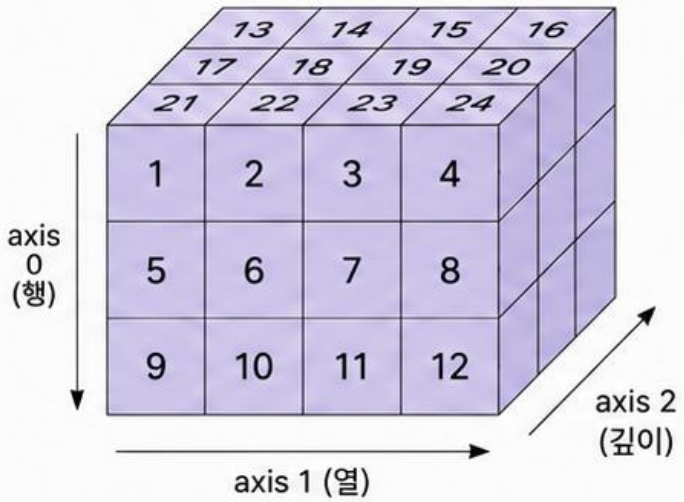
shape: (4,)

2-D array

5.2	3.0	4.5	1.2
9.1	0.1	0.3	7.7
2.2	8.8	6.0	1.5

shape: (3, 4)

3-D array



shape: (3, 4, 2)

3-D array 구조 해석

- axis 0 (행): 3
- axis 1 (열): 4
- axis 2 (깊이): 2

깊이 index 0

1	2	3	4
5	6	7	8
9	10	11	12

깊이 index 1

13	14	15	16
17	18	19	20
21	22	23	24

즉, (행, 열, 깊이) 순서로 shape가 결정됩니다.

ndarray attributes

3. 3차원 -> 4차원

```
import numpy as np
a = np.array([[[1,2],[3,4]],[[4,5,],[7,8]]])
print(a)
print(a.ndim)
```

```
[[[1 2]
  [3 4]]

 [[4 5]
  [7 8]]]
3
```

```
import numpy as np
a = np.array([[[[1,2],[3,4]],[[4,5,],[7,8]]]])
print(a)
print(a.ndim)
```

```
[[[[1 2]
   [3 4]]

  [[4 5]
   [7 8]]]]
4
```

Ndarray attributes

4. shape

- 배열의 각 차원의 크기
- 튜플의 형태로 리턴

```
arr
```

```
array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
[14] arr.ndim
```

```
1
```

```
[13] arr.shape
```

```
(8,)
```

```
arr2
```

```
array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]])
```

```
arr2.ndim
```

```
2
```

```
[17] arr2.shape
```

```
(2, 6)
```

ndarray 생성하기

1. array함수

- 기존에 있던 데이터(자료형)를 이용하여 새로운 배열을 생성

np.array(데이터, dtype=)

```
[2]  arr = np.array([10, 20, 30])
     arr
```

```
array([10, 20, 30])
```

→ 리스트를 이용하여 배열을 생성

```
[3]  arr2 = np.array([10, 20, 30], dtype=np.float16)
     arr2
```

```
array([10., 20., 30.], dtype=float16)
```

ndarray 생성하기

1. array함수

- 기존에 있던 데이터(자료형)를 이용하여 새로운 배열을 생성

```
[4]  arr3 = np.array(((1, 0), (0, 1)), dtype=np.float32)
     arr3
```

→ 튜플을 이용하여 배열을 생성

```
array([[1., 0.],
       [0., 1.]], dtype=float32)
```

```
[5]  arr4 = np.array(range(20))
     arr4
```

→ range 함수를 이용하여 배열을 생성

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

ndarray 생성하기

1. array함수

- 기존에 있던 데이터(자료형)를 이용하여 새로운 배열을 생성

```
[6]  arr5 = np.array([(10, 20), (40, 50)])
     arr5
```

```
array([[10, 20],
       [40, 50]])
```

```
[7]  arr6 = np.array(((1, 2), (3)))
     arr6
```

```
array([(1, 2), 3], dtype=object)
```

```
arr6.size
```

```
2
```

ndarray attributes

2. 배열 생성 함수

- numpy의 표준 배열 함수(참고)

- 자료형을 명시하지 않으면 float64

함수이름	설명
zeros	모두 0으로 초기화
ones	모두 1로 초기화
full	어떠한 값으로 모두 채워 초 기화
empty	초기화되지 않은 배열을 생 성
identity, eye	NxN 크기의 단위행렬
_likes	주어진 어떤 배열과 같은 shape의 배열을 생성
arange	range함수와 유사함, 범위와 stepsize
linspace	range함수와 유사함, sample 갯수

ndarray attributes

2. 배열 생성 함수

1) zeros / 모델 초기화나 임시 데이터 생성 등에 유용하게 쓰임

```
[16] np.zeros((5))
```

```
array([0., 0., 0., 0., 0.])
```

→ shape을 명시

```
[17] np.zeros((2, 4), dtype=np.int8)
```

```
array([[0, 0, 0, 0],
       [0, 0, 0, 0]], dtype=int8)
```

2) ones / 가중치를 초기화 할 때, 연산에서 모든 1인 배요ㅕㄹ이 필요할 때 주로 사용됨.

```
[18] np.ones((3,3))
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

ndarray attributes

2. 배열 생성 함수

3) full

```
[19] np.full((4), 5)

     array([5, 5, 5, 5])
```

```
     np.full((2, 5), -1.0)

     array([[-1., -1., -1., -1., -1.],
            [-1., -1., -1., -1., -1.]])
```

4) empty / 메모리 상태에 따라 결정되므로 어떤 값이 들어있는지 알 수 없습니다. 생성만 하고 초기화를 하지 않으므로, np.zeros, np.ones 보다 빠르게 실행될 수 있습니다.

```
[25] np.empty((2,3), dtype=np.float64)

     array([[1.8155276e-316, 0.0000000e+000, 0.0000000e+000],
            [0.0000000e+000, 0.0000000e+000, 0.0000000e+000]])
```

ndarray 생성하기

2. 배열 생성 함수

5) identity, eye (선형대수학에서 중요한 역할을 함.)

```
[22] np.identity(5, dtype=int)
```

→ N x N 정방행렬만 생성가능

```
array([[1, 0, 0, 0, 0],
       [0, 1, 0, 0, 0],
       [0, 0, 1, 0, 0],
       [0, 0, 0, 1, 0],
       [0, 0, 0, 0, 1]])
```

```
np.eye(5, dtype=int)
```

```
array([[1, 0, 0, 0, 0],
       [0, 1, 0, 0, 0],
       [0, 0, 1, 0, 0],
       [0, 0, 0, 1, 0],
       [0, 0, 0, 0, 1]])
```

```
np.eye(5, 10, dtype=int, k=5)
```

```
array([[0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]])
```

→ N x M 행렬도 생성가능

ndarray 생성하기

2. 배열 생성 함수

6) _like

- zeros_like, ones_like, full_like, empty_like

```
[29] arr1 = np.array([[1, 2, 3, 1], [2, 4, 5, 6]])
     arr1
```

```
array([[1, 2, 3, 1],
       [2, 4, 5, 6]])
```

```
arr2 = np.ones_like(arr1)
arr2
```

```
array([[1, 1, 1, 1],
       [1, 1, 1, 1]])
```

→ shape을 명시하지 않고 기존에 존재하는 배열을 인자로 넘겨줌

ndarray 생성하기

2. 배열 생성 함수

7) arange

- 파이썬 내장함수 range와 유사한 역할, ndarray를 반환

```
[43] np.arange(5)

     array([0, 1, 2, 3, 4])

[44] np.arange(-3, 3)

     array([-3, -2, -1,  0,  1,  2])

[45] np.arange(3,50,5)
     array([ 3,  8, 13, 18, 23, 28, 33, 38, 43, 48])
```

→ 세번째 인자는 step size (range와 동일)

ndarray attributes

2. 배열 생성 함수

8) linespace

- 범위내에서주어진sample의 갯수만큼 생성

Linspace(start, stop, num, endpoint ,retstep)

```
np.linspace(0, 1, 6)
```

```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

ndarray attributes

3. 배열 결합함수

1) Hstack , concatenate(axis =0)

- 두배열을 행 방향으로 연결되어 위아래로 붙임(수평 방향)

```
[7] a = np.array([1,2,3])

    b = np.array([4,5,6])
```

```
[8] np.hstack([a,b])

    array([1, 2, 3, 4, 5, 6])
```

```
[9] np.concatenate((a,b),axis =0)

    array([1, 2, 3, 4, 5, 6])
```

```
a = np.array([[1,2],[3,4]])

b = np.array([[5,6],[7,8]])

np.hstack([a,b])
```

```
array([[1, 2, 5, 6],
       [3, 4, 7, 8]])
```

```
np.concatenate((a,b),axis=0)
```

```
array([[1, 2],
       [3, 4],
       [5, 6],
       [7, 8]])
```

ndarray attributes

3. 배열 결합함수

2) vstack, concatenate(axis=1)

- 두 배열을 열 방향으로 연결되어 좌우로 붙임(수직방향)

```
np.vstack([a,b])
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.concatenate((a,b),axis =1)
```

```
---------------------------------------------------------------------------
AxisError                                 Traceback (most recent call last)
<ipython-input-111-5572ec4fb823> in <module>()
----> 1 np.concatenate((a,b),axis =1)

<__array_function__ internals> in concatenate(*args, **kwargs)

AxisError: axis 1 is out of bounds for array of dimension 1
```

1D 배열은 "AxisError: axis 1 is out of bounds for array of dimension 1"라는 AxisError가 나네요.
2D 이상 배열은 에러 없이 잘 됨.

```
c = np.array([[0, 1, 2], [3, 4, 5,]])
d = np.array([[6, 7, 8], [9, 10, 11]])

np.concatenate((c,d),axis =1)
```

```
array([[ 0,  1,  2,  6,  7,  8],
       [ 3,  4,  5,  9, 10, 11]])
```

ndarray attributes

3. 배열 결합함수

3) column_stack, concatenate(C.T,D.T, axis=1)

- 1차원 배열들을 각각 하나의 열로 만들어 좌우로 결합

```
[113] a,b

      (array([1, 2, 3]), array([4, 5, 6]))
```

```
[115] np.column_stack([a,b])

      array([[1, 4],
             [2, 5],
             [3, 6]])
```

1D 배열은 "AxisError: axis 1 is out of bounds for array of dimension 1"라는 AxisError가 나네요.
2D 이상 배열은 에러 없이 잘 됨.

```
c = np.array([[0, 1, 2], [3, 4, 5,]])
d = np.array([[6, 7, 8], [9, 10, 11]])

np.concatenate((c.T, d.T), axis = 1)
```

```
array([[ 0,  3,  6,  9],
       [ 1,  4,  7, 10],
       [ 2,  5,  8, 11]])
```

Random Module

- Random는 난수를 발생시키는 모듈. 모듈에는 여러 함수가 존재

```
import random
```

```
random.random() # 0<= 리턴값 <1.0
```

```
0.49262239384982O3
```

```
data = [1,2,3,4,5,6,7]

random.choice(data)
```

```
6
```

```
random.uniform(100, 200)# 괄호 안 두 수 사이의 실수 중에서 난수값을 리턴 a <= 실수 <= b
```

```
174.575900716115 48
```

```
data = ['apple', 'banana', 'grape', 'orange']

random.choice(data)
```

Choice 함수 : 매개변수로 시퀀스 타입(문자열, 튜플, 리스트)을 받음. 그 중에서 무작위로 하나를 선택하여 리턴

Random Module

- Random는 난수를 발생시키는 모듈. 모듈에는 여러 함수가 존재
- Seed : 난수 생성 초기값 부여
- 난수 생성 할 때 마다 값이 달라지는 것이 아니라, 누가 언제 하든지 간에 똑같은 난수 생성을 원한다면(즉, 재현가능성(Reproducibility)) seed 번호를 지정해줌.

- np.random.seed(100)

ndarray 생성하기

3. 난수 생성

- numpy.random 모듈을 이용하여 다양한 종류의 확률분포로부터 표본값을 생성 (참고)

함수이름	설명
np.random(size)	[0, 1) 균등 분포 난수
np.random.normal(loc, scale, size)	평균 loc, 표준편차 scale의 정규분포
np.random.integers(low, high, size)	Low 이상 high 미만의 정수 난수
np.random.randint( )	주어진 범위 내에서 임의의 난수를 추출
np.random.permutation(x)	순서를 임의로 바꾸거나 임의의 순열을 반환
np.random.shuffle(x)	리스트나 배열의 순서를 뒤섞음

ndarray 생성하기

3. 난수 생성

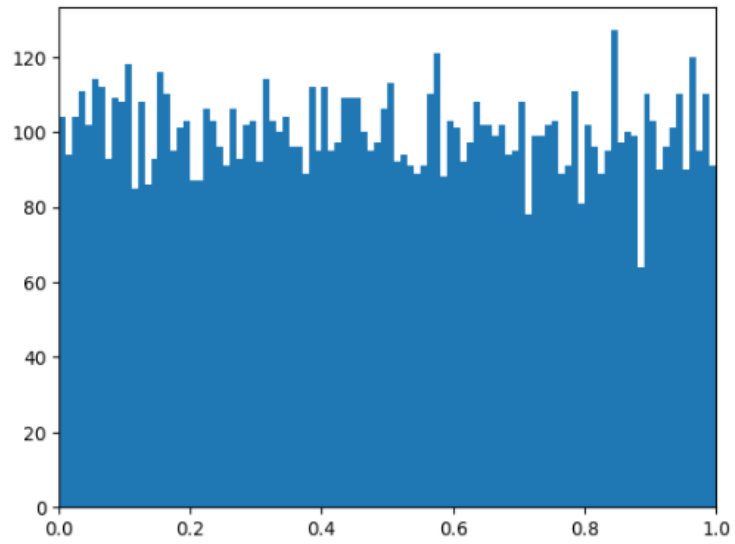
- numpy.random 모듈을 이용하여 다양한 종류의 확률분포로부터 표본값을 생성

```python
import numpy as np
import matplotlib.pyplot as plt

# 매번 다른 결과(진짜 랜덤처럼 보이게)
rng = np.random.default_rng()

# 0~1 사이 균등분포 난수
x = rng.random(10_000)

plt.figure()
plt.hist(x, bins=100, range=(0, 1))  # 그림처럼 거의 평평한 히스토그램
plt.xlim(0, 1)
plt.show()
```

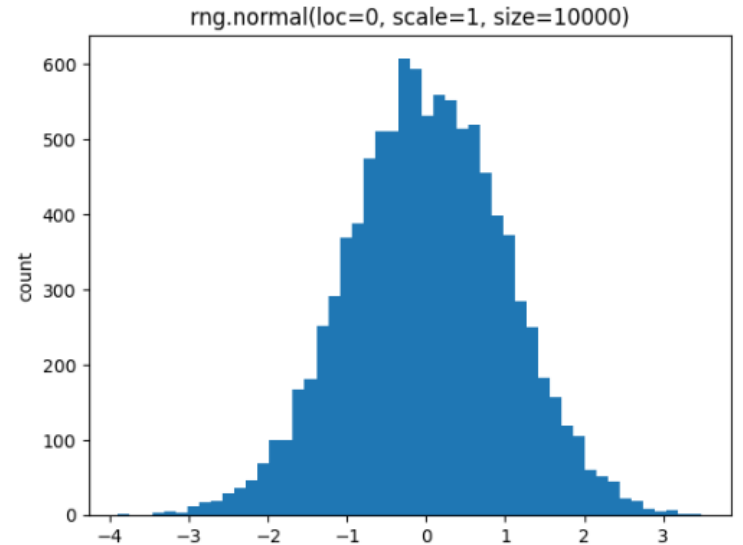



```python
import numpy as np
import matplotlib.pyplot as plt

# Reproducible RNG (recommended modern API)
rng = np.random.default_rng(0)

# 1) Normal distribution samples
x = rng.normal(loc=0.0, scale=1.0, size=10_000)

plt.figure()
plt.hist(x, bins=50)
plt.title("rng.normal(loc=0, scale=1, size=10000)")
plt.xlabel("value")
plt.ylabel("count")
plt.show()
```



ndarray 생성하기

```
#난수 생성
import numpy as np

#rand : 0과 1사이의 균일 분포에서 난수 생성
print("Random number between 0 and 1", np.random.rand())

#randn : 표준 정규 분포에서 난수 생성
print("Random number from standard normal distribution", np.random.randn())

#randint : 주어진 범위에서 임의의 정수 생성
print("Random integer between 1 and 10", np.random.randint(1,10))

#random : 0과 1 사이의 균일 분포에서 난수 생성(rand와 비슷하지만 다른 형태의 인자를 받음.)
print("Random number between 0 and 1 : ", np.random.random())
```

```
Random number between 0 and 1 0.0660969059688079
Random number from standard normal distribution 0.6425569110729781
Random integer between 1 and 10 9
Random number between 0 and 1 :  0.5490853188916499
```

ndarray 생성하기

```python
# 난수 생성 (최신 권장 방식: Generator API)
import numpy as np

# 시드 제외: 실행할 때마다 다른 난수
rng = np.random.default_rng()

# 0~1 사이 균등분포 난수 (기존: np.random.rand(), np.random.random())
print("Random number between 0 and 1:", rng.random())

# 표준 정규분포 난수 (기존: np.random.randn())
print("Random number from standard normal distribution:", rng.normal())

# 1~10 사이 정수 난수 (기존: np.random.randint(1, 10))
# 주의: integers는 high가 '미포함'이므로 10까지 포함하려면 high=11
print("Random integer between 1 and 10:", rng.integers(1, 11))
```

```
Random number between 0 and 1: 0.9742505843310069
Random number from standard normal distribution: 0.5345231200747645
Random integer between 1 and 10: 4
```

최신 권장 방식은 Generator(default_rng)를 사용하라고 권장

ndarray 생성하기

```
#랜덤 샘플링 : 주어진 집합에서 임의로 선택하는 과정

#choice : 주어진 배열에서 임의의 요소 선택
arr = np.array([1, 2, 3, 4, 5])
print("Random choice from array : ", np.random.choice(arr))

#shuffle : 배열을 무작위로 섞음 (원래 배열을 바꿈.)
np.random.shuffle(arr)
print('Array after shuffle', arr)

#permutation : 배열의 무작위 순열을 반환(원래 배열을 바꾸지 않음.)
print("Random permutatuion of array : ", np.random.permutation(arr))
```

```
Random choice from array :  1
Array after shuffle [4 3 5 2 1]
Random permutatuion of array :  [3 2 5 1 4]
```

연습 문제 1)

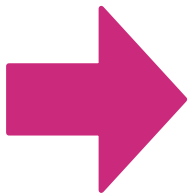
로또 번호 생성기를 만드세요.

```
로또 번호를 몇개 생성할까요?> 4
1. 로또번호: [13 16 20 22 27 43]
2. 로또번호: [21 27 31 36 37 45]
3. 로또번호: [ 4 15 16 26 30 39]
4. 로또번호: [ 7 15 26 29 40 41]
```

Flatten

N dim → 1 dim

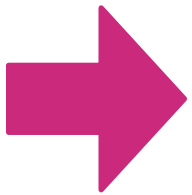
1	2	3
4	5	6
7	8	9



1	2	3	4	5	6	7	8	9

```
[60] arr = np.zeros((3,2))
     arr
```

```
array([[0., 0.],
       [0., 0.],
       [0., 0.]])
```



```
[61] arr.flatten()
```

```
array([0., 0., 0., 0., 0., 0.])
```

Reshape

- 이미 존재하는 배열을 내가 원하는대로 shape을 조정하는 함

np.reshape(arr, shape) arr.reshape(shape)

```
[76] arr = np.arange(12)
     arr
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
[77] arr.reshape(3, 4)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

주의

변환 전 후 배열의 전체 원소 수가 같아야 함.

Reshape

이미 존재하는 배열을 내가 원하는대로 shape을 조정하는 함수

```
[80] arr = np.arange(20)
     arr
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

```
[81] arr.reshape(-1, 10)
```

→ -1을 사용하면 shape을 명시하지 않아도 자동으로 채워줌

```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]])
```

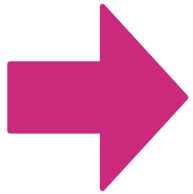
```
arr.reshape(5, -1)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19]])
```

Transpose

1	2	3
4	5	6
7	8	9

A



1	4	7
2	5	8
3	6	9

A^T

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

flatten() vs ravel() vs reshape(-1)

차원 축소(평탄화, 1차원 변환)할 때 사용하는 함수들의 차이를 이해하자!

함수	반환(Returned)
flatten()	항상 **복사본(copy)**을 반환
ravel()	가능하면 **view**를 반환, 필요하면 **복사본(copy)**을 반환
reshape(-1)	가능하면 **view**를 반환, 필요하면 **복사본(copy)**을 반환

핵심 포인트

- flatten()은 항상 새로운 배열을 만들기 때문에 원본과 메모리를 공유하지 않습니다.
- ravel()과 reshape(-1)은 가능한 경우 원본 데이터를 공유하는 **view**를 반환하여 메모리와 속도 축면에서 효율적입니다.
- 단, 배열의 메모리 배치(연속성, stride)에 따라 복사본이 반환될 수도 있습니다.

Tip

원본과 메모리를 공유하는지 확인하려면 **np.shares_memory(a, b)**를 사용합니다.

예제 코드 및 결과

예시 1) C-연속 배열 (기본 배열)

```
import numpy as np

a = np.arange(12).reshape(3, 4)

b1 = a.flatten()
b2 = a.ravel()
b3 = a.reshape(-1)


print(np.shares_memory(a, b1))  # False
print(np.shares_memory(a, b2))  # True
print(np.shares_memory(a, b3))  # True
```

결과 및 설명

함수	shares_memory	설명
flatten()	False	항상 복사본
ravel()	True	view 반환
reshape(-1)	True	view 반환

C-연속 배열에서는 ravel()과 reshape(-1)이 view를 반환합니다.

예시 2) 비연속 배열 (슬라이싱)

```
import numpy as np

a = np.arange(12).reshape(3, 4)
a_t = a.T  # 전치 -> 비연속 배열

b1 = a_t.flatten()
b2 = a_t.ravel()
b3 = a_t.reshape(-1)

print(np.shares_memory(a_t, b1))  # False
print(np.shares_memory(a_t, b2))  # False
print(np.shares_memory(a_t, b3))  # False
```

결과 및 설명

함수	shares_memory	설명
flatten()	False	항상 복사본
ravel()	False	복사본 반환
reshape(-1)	False	복사본 반환

비연속 배열에서는 ravel()과 reshape(-1)도 복사본를 반환할 수 있습니다.

정리

메모리 공유가 중요하고 배열이 C-연속적이라면 **ravel()** 또는 **reshape(-1)**을 사용하세요!
항상 독립된 복사본이 필요하다면 **flatten()**을 사용하세요!

ravel vs flatten vs reshape

```python
import numpy as np

a = np.arange(12).reshape(3, 4)
print("a:₩n", a)
print("a.shape:", a.shape)

# 1) ravel: 가능한 경우 view(원본 공유), 아니면 copy
r = a.ravel()

# 2) flatten: 항상 copy(원본과 독립)
f = a.flatten()

# 3) reshape(-1): 가능한 경우 view, 아니면 copy (ravel과 유사한 성격)
s = a.reshape(-1)

print("₩n--- 기본 비교 ---")
print("ravel   ->", r.shape, "shares_memory:", np.shares_memory(a, r))
print("flatten ->", f.shape, "shares_memory:", np.shares_memory(a, f))
print("reshape ->", s.shape, "shares_memory:", np.shares_memory(a, s))

# 원본을 바꿔서 확인
a[0, 0] = 999
print("₩n원본 a[0,0]=999 후")
print("ravel[0]  :", r[0], "(원본과 공유하면 999로 바뀜)")
print("flatten[0]:", f[0], "(항상 copy라 원래 값 유지)")
print("reshape[0]:", s[0], "(공유하면 999로 바뀜)")

# ravel/reshape 결과를 바꿔서 원본이 바뀌는지 확인
r[1] = 777
print("₩nravel[1]=777 후 원본 a:₩n", a)

# ---- reshape가 view를 못 만드는 대표 케이스: 비연속 배열 ----
b = a[:, ::2]   # stride가 생겨 비연속일 수 있음
print("₩n₩nb = a[:, ::2] (비연속 가능) ₩n", b)
print("b.flags['C_CONTIGUOUS']:", b.flags['C_CONTIGUOUS'])

br = b.ravel()
bf = b.flatten()
bs = b.reshape(-1)  # 이 경우 copy가 될 가능성이 큼

print("₩n--- 비연속 배열에서 비교 ---")
print("b.ravel   shares_memory:", np.shares_memory(b, br))
print("b.flatten shares_memory:", np.shares_memory(b, bf))
print("b.reshape shares_memory:", np.shares_memory(b, bs))

# 비연속 케이스에서 원본 변경으로 확인
b[0, 0] = -1
print("₩nb[0,0]=-1 후")
print("br[0]:", br[0], "(공유면 -1)")
print("bf[0]:", bf[0], "(항상 copy)")
print("bs[0]:", bs[0], "(공유면 -1, copy면 기존값)")
```

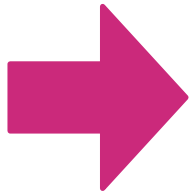
shape_numpy.py

Transpose

np.transpose(arr, axes)
arr.transpose(axes)

```
[83] arr = np.arange(20).reshape(4, 5)
     arr
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```



```
[84] print(arr.transpose().shape)
     arr.transpose()
```

```
(5, 4)
array([[ 0,  5, 10, 15],
       [ 1,  6, 11, 16],
       [ 2,  7, 12, 17],
       [ 3,  8, 13, 18],
       [ 4,  9, 14, 19]])
```

Transpose

- axes를 지정하지 않으면?

  a.shape = (i[0], i[1], ⋯, i[n-1]) -> a^T.shape = (i[n-1], i[n-2], ⋯,i[0])

```
[2]  arr = np.arange(30).reshape(3,2,5)
     arr.shape
```

```
(3, 2, 5)
```

```
[3]  print(arr.transpose().shape)
```

```
(5, 2, 3)
```

Swapaxes

- np.swapaxes는 직관적으로 축을 선정 함.

```
import numpy as np

a = np.arange(3).reshape(1,3) # (1X3) 2D array

y = np.swapaxes(a,0,1) # 0은 가장 높은 차수의 축, 2차원 / 1은 그다음 높은 차수의 축 1차원 즉, 원소의 행과 열을 바꾸라는 것
y
```

```
array([[0],
       [1],
       [2]])
```

Swapaxes_차원의 증가

```
a = np.arange(6).reshape(1,2,3)
print(a)
print(a.shape)
```

```
[[[0 1 2]
  [3 4 5]]]
(1, 2, 3)
```

```
x = np.swapaxes(a, 1, 2) # 2차원 축과, 1차원 축을 바꿔라
x
```

```
array([[[0, 3],
        [1, 4],
        [2, 5]]])
```

```
y = np.swapaxes(a, 0, 1) # 3차원 축과, 2차원 축을 바꿔라.
print(y)
print(y.shape)
```

```
[[[0 1 2]]

 [[3 4 5]]]
(2, 1, 3)
```

```
z = np.swapaxes(a, 0, 2) # 3차원 축과 1차원 축을 바꿔라.
print(z)
print(z.shape)
```

```
[[[0]
  [3]]

 [[1]
  [4]]

 [[2]
  [5]]]
(3, 2, 1)
```

Transpose

- axes는 tuple이나 list로 지정해줄수 있음

```
[4]  print(arr.shape)
     arr.transpose((1, 0, 2)).shape
```

```
(3, 2, 5)
(2, 3, 5)
```

```
arr = np.arange(120).reshape(2,3,4,5)
print(arr.shape)
arr.transpose((1, 0, 2, 3)).shape
```

```
(2, 3, 4, 5)
(3, 2, 4, 5)
```

T operation

- Transpose와 같은 역할을 하는 ndarray의 attribute
- 단, T의 경우 axes를 지정할 수 없음

```
[7]  x = np.arange(6).reshape((-1,3))
     x
```

```
array([[0, 1, 2],
       [3, 4, 5]])
```

```
x.T
```

```
array([[0, 3],
       [1, 4],
       [2, 5]])
```

연습문제 2)

0~ 20까지의 숫자를 배열을 만든 다음에 arr1에는 짝수,

arr2는 홀수가 들어간 배열을 출력해보자.

연습문제 3)

주어진 배열을 이용하여 아래와 같은 결과를 만들어보자

```
arr = np.arange(30).reshape(2,3,5)
arr
```

```
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14]],

       [[15, 16, 17, 18, 19],
        [20, 21, 22, 23, 24],
        [25, 26, 27, 28, 29]]])
```

```
array([[ 0,  5, 10, 15, 20, 25],
       [ 1,  6, 11, 16, 21, 26],
       [ 2,  7, 12, 17, 22, 27],
       [ 3,  8, 13, 18, 23, 28],
       [ 4,  9, 14, 19, 24, 29]])
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29])
```

Indexing & Slicing

- 1차원

기존의 리스트와 유사함

```
[9]  arr1d = np.arange(8)
     arr1d
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
[10] arr1d[1]
```

```
1
```

```
[11] arr1d[-1]
```

```
7
```

```
[12] arr1d[:4]
```

```
array([0, 1, 2, 3])
```

Indexing & Slicing

- 1차원

ndarray vs. list

```
[15] lst = list(range(6))
     lst
```

```
[0, 1, 2, 3, 4, 5]
```

```
[16] lst[2:5] = -1
     lst
```

```
----------------------------------------
TypeError
<ipython-input-16-b4d2405268a4> in <modu
----> 1 lst[2:5] = -1
      2 lst

TypeError: can only assign an iterable
```

```
[17] lst[3] = -1
     lst
```

```
[0, 1, 2, -1, 4, 5]
```

→ 브로드캐스팅을 지원하지 않음

Indexing & Slicing

- 1차원

  ndarray vs. list

```
[9]  arr1d = np.arange(8)
     arr1d
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
[13] arr1d[3:6] = 100
```

→ 브로드캐스팅을 지원

```
arr1d
```

```
array([  0,   1,   2, 100, 100, 100,   6,   7])
```

Indexing & Slicing

- 1차원

  Ndarray VS list

2) View

Numpy의 Slicing은 원본데이터의 뷰를 제공

: 데이터를 새롭게 복사해 오는 것이 아니라 기존의 데이터와 연결되어 있음

리스트의 슬라이싱은 데이터를 복사하게 됨

Indexing & Slicing

- 1차원

  ndarray vs. list

```
[17] arr_part = arr1d[:3]
     arr_part
```

```
array([0, 1, 2])
```

→ 슬라이싱

```
[18] arr_part[1:] = -1
     arr_part
```

```
array([ 0, -1, -1])
```

→ 값을 변경

```
[19] arr1d
```

```
array([  0,  -1,  -1, 100, 100, 100,   6,   7])
```

→ 원본데이터가 변경되어있음

Indexing & Slicing

- 1차원

  ndarray vs. list

```
[20] lst_part = lst[2:]
     lst_part
```

```
[2, -1, 4, 5]
```

→ 슬라이싱

```
[21] lst_part[3] = 100
     lst_part
```

```
[2, -1, 4, 100]
```

→ 값을 변경

```
lst
```

```
[0, 1, 2, -1, 4, 5]
```

→ 원본데이터는 그대로 유지

Indexing & Slicing

- 1차원

  ndarray vs. list

  2) 뷰

  - 원본데이터를훼손하지 않으려면?

    arr.copy( )

```
[27] new_arr = arr1d.copy()
     new_arr
```

```
array([  0,  -1,  -1, 100, 100, 100,   6,   7])
```

Indexing & Slicing

- 다차원

```
[3]  arr2d = np.arange(20).reshape(4, -1)
     arr2d
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
[4]  arr2d[0]
```

→ 다중리스트의 인덱싱과 유사함

```
array([0, 1, 2, 3, 4])
```

```
[5]  arr2d[1][2]
```

→ 재귀적으로 접근

```
7
```

```
[6]  arr2d[1, 2]
```

→ 콤마(,)를 이용하여 쉽게 인덱싱을 할 수도 있음

```
7
```

Indexing & Slicing

- 다차원

```
arr2d = np.arange(20).reshape(4, -1)
arr2d
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
arr2d[:3][:2]
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

→ 재귀적으로 슬라이싱을 함

```
arr2d[:3,:2]
```

```
array([[ 0,  1],
       [ 5,  6],
       [10, 11]])
```

Boolean indexing

- 불리언 배열 : 불리언 값으로 이루어진 배열

```
[2]  arr = np.array([True, False])
     arr.dtype
```

```
dtype('bool')
```

- 불리언 인덱싱 : 불리언 배열을 이용한 인덱싱

  - True에 해당되는 위치에 있는 값만을 반환

```
[3]  arr = np.array([0, 1, 2, 3, 4], int)
     arr[[True, False, True, False, True]]
```

```
array([0, 2, 4])
```

Boolean indexing

- 불리언 배열을 이용한 인덱싱

```
[4]  arr = np.array([10, 20, 30, 40, 50, 60], int)
     arr
```

```
array([10, 20, 30, 40, 50, 60])
```

```
[5]  arr % 3 == 0
```

```
array([False, False,  True, False, False,  True])
```

```
arr[arr%3==0]
```

```
array([30, 60])
```

→ 조건문을 통해 불리언 배열을 만들 수 있음

→ 이를 배열에 인덱싱으로 주면 해당 위치에 값을 반환

- 불리언 인덱싱 후 해당 값을 다시 인덱싱/슬라이싱도 할 수 있음

Fancy indexing

정수 배열을 사용한 인덱싱

```
[5]  arr = np.arange(10, 20)
     arr
```

```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
[6]  arr[[0, 2, 4, 6]]
```

```
array([10, 12, 14, 16])
```

```
arr[[3, 0, 1]]
```

```
array([13, 10, 11])
```

→ 정수가 담긴 ndarray나 리스트로 특정 위치에 있는 값을 가져올 수 있음

→ 주어지는 순서대로 값을 가져오게 됨

Fancy indexing

정수 배열을 사용한 인덱싱

```
[7]  arr2d = np.arange(20).reshape(4, 5)
     arr2d
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
[8]  arr2d[[0, 2]]
```

```
array([[ 0,  1,  2,  3,  4],
       [10, 11, 12, 13, 14]])
```

```
[9]  arr2d[[0, 1], [4]]
```

```
array([4, 9])
```

Fancy indexing

정수 배열을 사용한 인덱싱

```
[7]  arr2d = np.arange(20).reshape(4, 5)
     arr2d
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
[10] arr2d[[0, 1], [4, 3]]
```

→ (0, 4), (1, 3)에 대응되는 원소들을 가져옴

```
array([4, 8])
```

```
arr2d[[0, 1, 2], [4, 3, 1]]
```

```
array([ 4,  8, 11])
```

Fancy indexing

정수 배열을 사용한 인덱싱

```
[7]  arr2d = np.arange(20).reshape(4, 5)
     arr2d
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

```
arr2d[[0, 1, 2]] [:, [4, 3, 1]]
```

```
array([[ 4,  3,  1],
       [ 9,  8,  6],
       [14, 13, 11]])
```

연습문제 3)

1번

주어진 배열을 이용하여 아래와 같은 결과를 만들어보자

```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8],
        [ 9, 10, 11]],

       [[12, 13, 14],
        [15, 16, 17],
        [18, 19, 20],
        [21, 22, 23]]])
```

2번

주어진 배열과 불리언 인덱싱을 이용하여 아래와 같은 결과를 만들어보자

```
arr = np.arange(30).reshape(3,2,5)
arr
```

```
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9]],

       [[10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]],

       [[20, 21, 22, 23, 24],
        [25, 26, 27, 28, 29]]])
```

```
cond = np.array(['a', 'b', 'c'])
```

```
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9]],

       [[20, 21, 22, 23, 24],
        [25, 26, 27, 28, 29]]])
```

유니버설 함수

- Ndarray 안에 있는 데이터 원소별로 연산을 수행하는 함수
- 하나 이상의 스칼라값을 받아서 하나 이상의 스칼라값을 반환하는 간단한 함수를 고속으로 수행할 수 있는 벡터화된 함수

Function	설명
abs, fabs	각 원소의 절대값. 복소수가 아닌 경우 fabs를 쓰면 빠른 연산이 가능
sqrt	각 원소의 제곱근 계산(arr**0.5)
square	각 원소의 제곱을 계산(arr**2)
exp	각 원소에 지수e^x 계산
log, log10, log2, log1p	자연로그, 밑10인 로그, 밑2인 로그, log(1+x)
sign	각 원소의 부호(양수 1, 영 0, 음수 -1)
ceil	각 원소의 값보다 같거나 큰 정수 중 가장 작은 값
floor	각 원소의 값보다 같거나 작은 정수 중 가장 큰 값
rint	각 원소의 소수자리를 반올림
modf	각 원소의 몫과 나머지를 각각의 배열로 반환
isnan	각 원소가 NaN인지 아닌지. 불리언 배열로 반환
isfinite, isinf	각 원소가 유한한지, 무한한지. 불리언 배열로 반환
cos, cosh, sin, sinh, tan, tanh	일반 삼각함수, 쌍곡삼각함수
arccos, arccosh, arcsin, arcsinh, arctan, arctanh	역삼각함수

유니버설 함수(ufunc)

- 이항유니버셜함수

Function	설명
add	두 배열의 같은 위치의 원소끼리 합함
subtract	첫번째 배열의 원소에서 두번째 배열의 원소를 뺌
multiply	같은 위치의 원소끼리 곱함
divide, floor_divide	첫번째 배열의 원소를 두번째 배열의 원소를 나눔. floor는 몫만 취함
power	첫번째 매열의 원소를 두번째 배열의 원소만큼 제곱함
maximum, fmax	각 배열의 두 원소 중 큰 값을 반환. fmax는 NaN 무시
minimum, fmin	각 배열의 두 원소 중 작은 값을 반환. fmax는 NaN 무시
mod	첫번째 배열의 원소를 두번째 배열의 원소로 나눈 나머지
copysign	첫번째 배열의 원소의 기호를 두번째 배열의 원소의 기호로 바꿈
greater, greater_equal, less, less_equal, less, less_equal,	각 두 원소 간의 비교 연산(<, <=, >, >=, ==, !=)를 불리언 배열로 반환

유니버설 함수(ufunc)

- 이항유니버셜함수

```
[6]  arr1 = np.arange(8).reshape(2, -1)
     arr2 = np.arange(-40, 40, 10).reshape(2, -1)
     print(arr1)
     print(arr2)
```

```
[[0 1 2 3]
 [4 5 6 7]]
[[-40 -30 -20 -10]
 [  0  10  20  30]]
```

```
[7]  np.maximum(arr1, arr2)
```

```
array([[ 0,  1,  2,  3],
       [ 4, 10, 20, 30]])
```

```
[9]  np.subtract(arr2, arr1)
```

```
array([[-40, -31, -22, -13],
       [ -4,   5,  14,  23]])
```

```
[10] np.multiply(arr1, arr2)
```

```
array([[  0, -30, -40, -30],
       [  0,  50, 120, 210]])
```

기본 통계 메소드

- 배열 전체 혹은 배열에서 한 축을 따르는 자료에 대한 통계를 계산하는 수학 함수

sum
mean
std, var
min, max
argmin, argmax
cumsum
cumprod

```
[11] arr = np.random.randn(200, 500)
     arr.shape

     (200, 500)

[12] arr.sum()    # ndarray의 메서드 이용

     -596.3978294521196

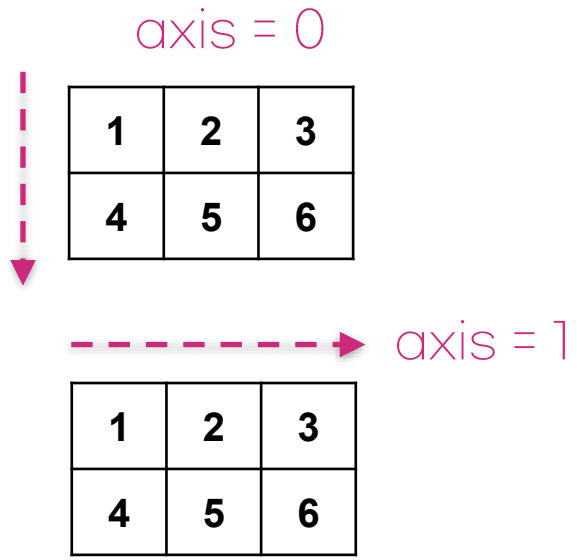
[13] np.sum(arr)     # numpy의 최상위 함수를 이용

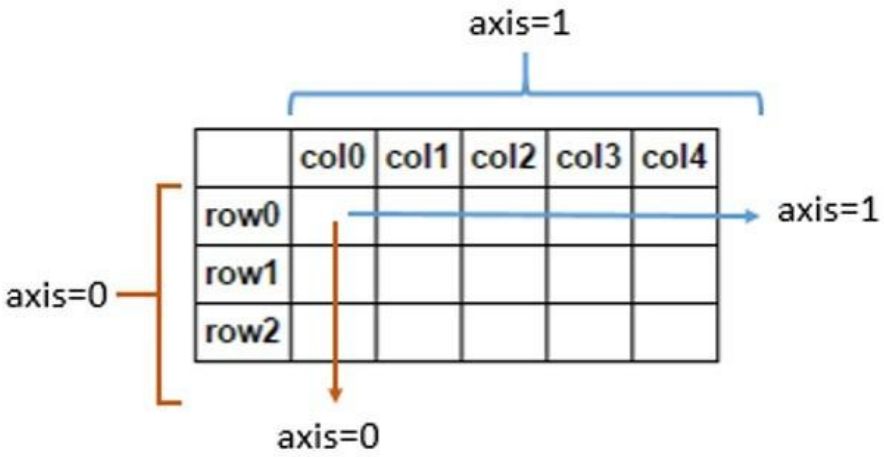
     -596.3978294521196
```

기본 통계 메소드

- sum, mean과 같은 함수의 경우 axis를 인자로 받아서 해당 axis에 대한 통계를 계산할 수 있음
- 축을 지정하지 않을 경우 배열 전체에 대한 값은 연산

```
[21] arr.shape
     (200, 500)
[14] arr.mean()
     -0.005963978294521196
[22] arr.mean(axis=0).shape
     (500,)
[23] arr.mean(axis=1).shape
     (200,)
```





Numpy의 메모리 접근 패턴과 캐시 최적화

- Numpy 배열은 메모리에 연속적으로 저장되며, 행 우선(row-major) 저장 방식을 사용합니다.
- 이는 CPU 캐시 활용을 극대화하여 연산 속도를 향상시키며, 특히 행 기반 접근이 열 기반 접근보다 더 빠르게 동작합니다.
- 따라서 큰 데이터를 처리할 때 효과적인 데이터 조작과 연산 성능 향상이 가능합니다.

```
import time

#2차원 배열 생성
arr = np.random.rand(10000,10000)

#열의 합을 계산
start = time.time()
column_sum = np.sum(arr, axis=0)
print("Time for column sum : ", time.time() - start)

#행의 합을 계산
start = time.time()
row_sum = np.sum(arr,axis=1)
print("Time for row sum", time.time() - start)
```

기본 통계 메소드

표본 분산/표준편차도 계산 가능

$$s^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2$$

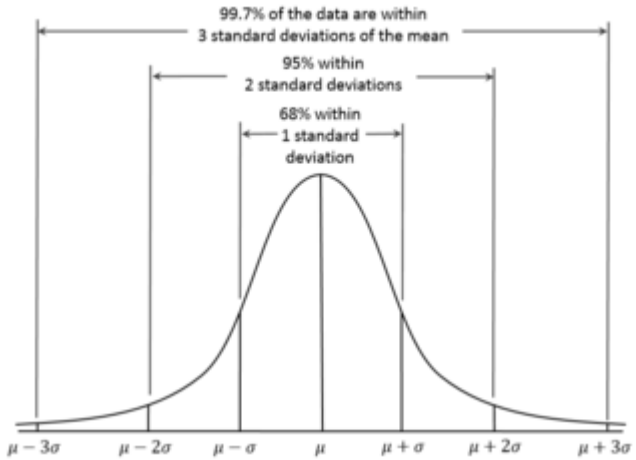
$$s = \sqrt{s^2}$$

```
np.var(x)  # 분산
```

```
115.23224852071006
```

```
np.std(x)  # 표준 편차
```

```
10.734628476137871
```



Numpy의 rand/randn

```
import numpy as np

np.random.rand()
```

```
0.1454858837063564
```

```
np.random.rand(1)
```

```
array([ 0.36708798])
```

```
np.random.rand(2)
```

```
array([ 0.94584748,  0.10367624])
```

```
np.random.rand(3,3)
```

```
array([[ 0.52603453,  0.54570411,  0.99713388],
       [ 0.11226014,  0.64858858,  0.96787497],
       [ 0.76334219,  0.90901883,  0.48988668]])
```

```
np.random.rand(5,2)
```

```
array([[ 0.78907522,  0.8219071 ],
       [ 0.84726884,  0.50501422],
       [ 0.26994445,  0.90803876],
       [ 0.35450963,  0.21855857],
       [ 0.43977184,  0.48358754]])
```

기타 메소드_any,all

- any
  - 하나 이상의 값이 True이면 True를 반환
- all
  - 모든값이 True 이면 True를 반환

```
[2]  arr = np.array([True, False, True])
     arr.any()
```

```
True
```

```
[3]  arr = np.array([True, False, True])
     arr.all()
```

```
False
```

```
[4]  arr = np.array([0, 0, 3])
     arr.any()
```

```
True
```

```
[5]  arr = np.array([0, 0, 1])
     arr.all()
```

```
False
```

기타 메소드_where

- Where
  - x if 조건 else y의 벡터화 버전
  - numpy를 사용하여 큰 배열을 빠르게 처리 할 수 있으며, 다차원도 간결하게 표현 가능

```
[8]  xarr = np.array([100, 200, 300, 400])
     yarr = np.array([1, 2, 3, 4])
     cond = np.array([True, False, True, False])
```

→ True 일때는 xarr에 있는 값을,
False 일때는 yarr에 있는 값을 대입하여 반환

```
[9]  result = np.where(cond, xarr, yarr)
     result
```

```
array([100,   2, 300,   4])
```

기타 메소드_where

where

- x와 y자리에 들어가는 인자는 배열이 아니여도 가능

```
[8]  xarr = np.array([100, 200, 300, 400])
     yarr = np.array([1, 2, 3, 4])
     cond = np.array([True, False, True, False])
```

```
[10] np.where(xarr>200, max(xarr), 0)
```

```
array([  0,   0, 400, 400])
```

```
np.where(xarr%3==0, 1, 0 )
```

```
array([0, 0, 1, 0])
```

기타 메소드_Sort

- Sort
  - arr.sort()
    - 주어진 축에 따라 정렬하며, 다양한 정렬방법들을 지원
    - arr자체를 정렬함(in-place)

```
[23] np.random.seed(10)
     arr = np.random.randint(1, 100, size=10)
     arr
```

```
array([10, 16, 65, 29, 90, 94, 30,  9, 74,  1])
```

```
     arr.sort()
     arr
```

```
array([ 1,  9, 10, 16, 29, 30, 65, 74, 90, 94])
```

기타 메소드_Sort

- Sort
  - np.sort()
    - 배열을 직접 변경하지 않고, 정렬된 결과를 가진 복사본을 반환

np.sort(arr, axis=-1)

```
[32] np.random.seed(20)
     arr = np.random.randint(1, 100, size=10)
     np.sort(arr)
```

```
array([10, 16, 21, 23, 29, 72, 76, 91, 91, 96])
```

```
[33] arr
```

```
array([91, 16, 96, 29, 91, 10, 21, 76, 23, 72])
```

```
-np.sort(-arr)
```

→ 부호를 이용하여 내림차순으로 정렬

```
array([96, 91, 91, 76, 72, 29, 23, 21, 16, 10])
```

선형대수

- numpy는 행렬의곱셈, 분할, 행렬식과같은선형대수에관한라이브러리를 제공함

Functions	설명
dot	내적
diag	2차원 배열에서 지정한 대각선의 원소를 추출하거나, 1차원 배열의 값을 대각 원소로 갖는 대각 행렬을 생성
trace	행렬의 대각선 원소의 합을 계산
linalg.det	행렬식을 계산(ad-bc)
linalg.eig	정사각행렬의 고유값, 고유벡터를 계산
linalg.inv	정사각행렬의 역행렬을 계산
linalg.solve	A가 정사각 행렬일 때, Ax=b를 만족하는 x를 구함
linalg.svd	특이값 분해(SVD)를 계산

선형 대수_단위 행렬

- 단위 행렬(Identity Matrix)
  - 단위 행렬은 대각원소가 1이고, 나머지는 모두 0인 n차 정방행렬을 말하며, numpy의 eye() 함수를 사용하여서 만들 수 있습니다.

```
import numpy as np

Identity = np.eye(4)
print(Identity)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
import numpy as np

Identity = np.eye(2,3)
print(Identity)
```

```
[[1. 0. 0.]
 [0. 1. 0.]]
```

선형 대수_대각행렬

- 대각 행렬(Diagonal Matrix)
  - 대각행렬은 대각성분 이외의 모든 성분이 모두 '0'인 n차 정방행렬을 말함.

```
import numpy as np

x = np.arange(9).reshape(3,-1)

print(x)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```
np.diag(x)
```

```
array([0, 4, 8])
```

```
np.diag(np.diag(x))
```

```
array([[0, 0, 0],
       [0, 4, 0],
       [0, 0, 8]])
```

선형 대수_dot

- dot
  - dot product : 벡터의 내적

np.dot(a, b)

$$\mathbf{a} = (a_1, a_2, \ldots, a_n)$$

$$\mathbf{b} = (b_1, b_2, \ldots, b_n)$$

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

선형 대수_dot

원소 간 곱(element-wise product) :
a*b

```
import numpy as np

a = np.arange(4).reshape(-1,2)
print(a)
a*a
```

```
[[0 1]
 [2 3]]
array([[0, 1],
       [4, 9]])
```

내적(dot product, inner product):
np.dot(a,b)

```
import numpy as np

a = np.arange(4).reshape(-1,2)
print(a)
print(np.dot(a,a))
```

```
[[0 1]
 [2 3]]
[[ 2  3]
 [ 6 11]]
```

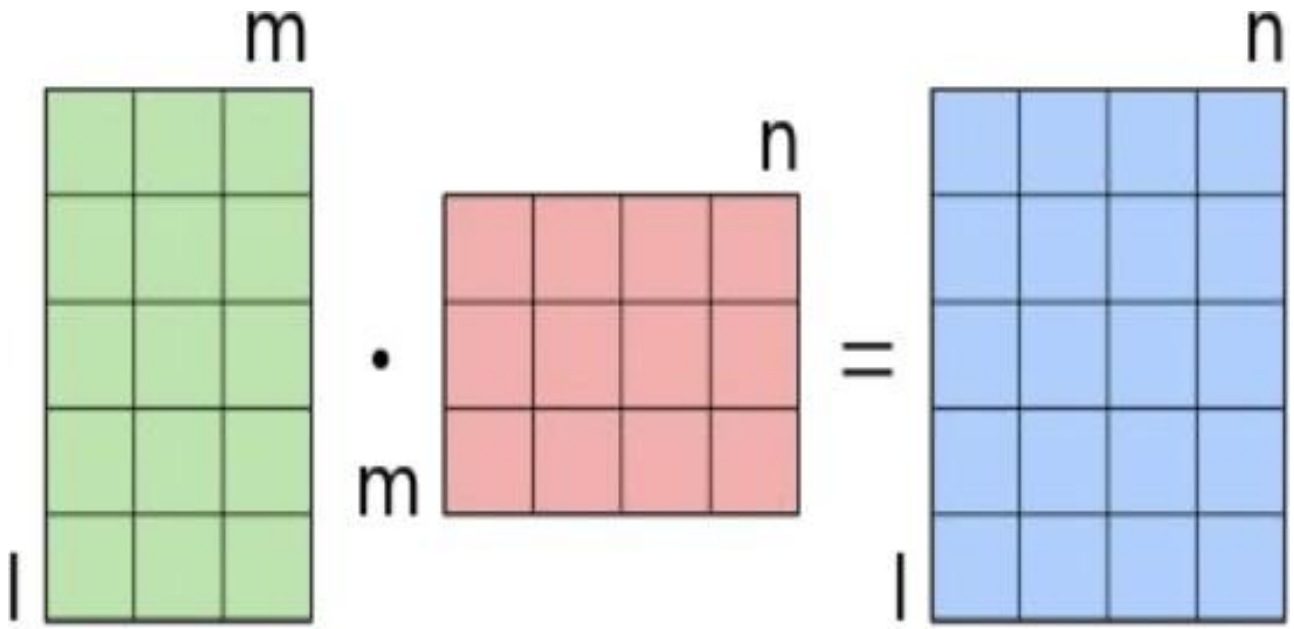
동일한 작동

```
a.dot(a)
```

```
array([[ 2,  3],
       [ 6, 11]])
```

선형 대수_matmul

- matmul
  - matrix multiplication : 행렬의곱



선형 대수_matmul

- matmul : matrix multiplication : 행렬의곱

np.matmul(a, b)

```
[41] a = np.random.randint(-3, 3, 10).reshape(2, 5)
     b = np.random.randint(0, 5, 15).reshape(5, 3)
     a.shape, b.shape
```

```
((2, 5), (5, 3))
```

```
[42] ab = np.matmul(a, b)
     print(ab.shape,'\n')
     print(ab)
```

```
(2, 3)

[[  6   0   6]
 [-14  -6  -4]]
```

선형 대수_matmul

- Matmul : matrix multiplication : 행렬의곱

np.matmul(a, b)

```
[41] a = np.random.randint(-3, 3, 10).reshape(2, 5)
     b = np.random.randint(0, 5, 15).reshape(5, 3)
     a.shape, b.shape

     ((2, 5), (5, 3))

     np.matmul(b, a)
```

→ matmul시에는 shape에 유의할 것

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-43-3c9936ad69e2> in <module>()
----> 1 np.matmul(b, a)  # matrix곱할때는 shape이 맞도록

ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0,
            with gufunc signature (n?,k),(k,m?)->(n?,m?) (size 2 is different from 3)
```

선형 대수_matmul

- matmul
  - 3차원 이상의 경우에는 **마지막2개** 축으로 이루어진 행렬을 다른 축들에 따라쌓은 것으로 파악
  - 따라서, **마지막 2개의 차원이 행렬곱을 할 수 있으면** matmul 가능

```
[44] c = np.arange(24).reshape(2, 3, 4)
     d = np.arange(2*4*5).reshape(2, 4, 5)
     c.shape, d.shape
```

```
((2, 3, 4), (2, 4, 5))
```

```
[48] arr = np.matmul(c, d)
     arr.shape
```

```
(2, 3, 5)
```

선형 대수_대각합(Trace)

- Trace
  - 2차원 배열의 지정된 대각성 원소의 합을 계산. 정방행렬이 아니어도 사용이 가능

```
import numpy as np

b = np.arange(16).reshape(4,-1)

print(b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
np.trace(b)
```

```
30
```

```
c = np.arange(27).reshape(3, 3, 3)
print(c)
```

```
[[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[ 9 10 11]
  [12 13 14]
  [15 16 17]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]
```

```
np.trace(c)
```

```
array([36, 39, 42])
```

선형 대수_행렬식(Matrix Determinant)

- Determinant

  - 역행렬이 존재하는지 여부를 확인하는 방법으로 행렬식(determinant, 줄여서 det)이라는 지표를 사용함. 이 행렬식이 '0'이 아니면 역행렬이 존재하고, 이 행렬식이 '0'이면 역행렬이 존재하지 않습니다.

```
d = np.array([[1,2],[3,4]])

np.linalg.det(d)
```

```
-2.0000000000000004
```

```
d = np.array([[1,2],[2,4]])

np.linalg.det(d)
```

```
0.0
```

선형 대수_역행렬(Inverse of a matrix)

- Inverse matrix

- 역행렬은 n차 정방행렬 A: $\mathbf{n} \times \boldsymbol{n}$과의 곱이 항등행렬 또는 단위 행렬이 되는 n차 정방 행렬을 말합니다.

A*B와 B*A 모두 순서에 상관없이 곱했을 때 단위행렬이 나오는 n차 정방행렬이 있다면 역행렬이 존재함. 역행렬은 가우스 소거법(Gauss-Jordan elimination method)혹은 여인수(Cofactor method)로 풀 수 있음.

```
a = np.array(range(4)).reshape(2, -1)
print(a)
```

```
[[0 1]
 [2 3]]
```

```
a_inv = np.linalg.inv(a)
```

```
a_inv
```

```
array([[-1.5,  0.5],
       [ 1. ,  0. ]])
```

```
a.dot(a_inv)
```

```
array([[1., 0.],
       [0., 1.]])
```

선형 대수_고유 값(Eigenvalue), 고유벡터(Eigenvector)

정방 행렬 A에 대하여 $Ax = \lambda x$(상수 λ)가 성립하는 0이 아닌 벡터 x가 존재할 때 상수 λ를 행렬 A의 고유 값(eigenvalue), x를 이에 대응하는 고유 벡터(eigenvector)라고 함.

```
e = np.array([[4, 2],[3, 5]])
print(e)
```

```
[[4 2]
 [3 5]]
```

```
w,v = np.linalg.eig(e)
```

```
print(w)
```

```
[2. 7.]
```

```
print(v)
```

```
[[-0.70710678 -0.5547002 ]
 [ 0.70710678 -0.83205029]]
```

선형 대수_특이값 분해(Singular Value Decomposition)

특이값 분해는 고유값 분해(eigen decomposition)처럼 행렬을 대각화하는 한 방법으로서, 정방행렬뿐만 아니라 모든 m x n 행렬에 대해 적용 가능합니다. 특이값 분해는 차원축소, 데이터 압축 등에 사용할 수 있습니다.

```
A = np.array([[3,6], [2,3], [0,0], [0,0]])

print(A)
```

```
[[3 6]
 [2 3]
 [0 0]
 [0 0]]
```

```
u,s,vh = np.linalg.svd(A)
```

```
print(u)
```

```
[[-0.8816746  -0.47185793  0.          0.        ]
 [-0.47185793  0.8816746   0.          0.        ]
 [ 0.          0.          1.          0.        ]
 [ 0.          0.          0.          1.        ]]
```

```
print(s)
```

```
[7.60555128 0.39444872]
```

```
print(vh)
```

```
[[-0.47185793 -0.8816746 ]
 [ 0.8816746  -0.47185793]]
```

Thank you.

Numpy
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.